

Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Навчально-науковий Фізико-Техічний інститут
Кафедра інформаційної безпеки

Лабораторна робота №3
з дисципліни «БЛОКЧЕЙН ТА ДЕЦЕНТРАЛІЗОВАНІ СИСТЕМИ»

Виконав:
ст. групи ФБ-52мп Слепий Роман

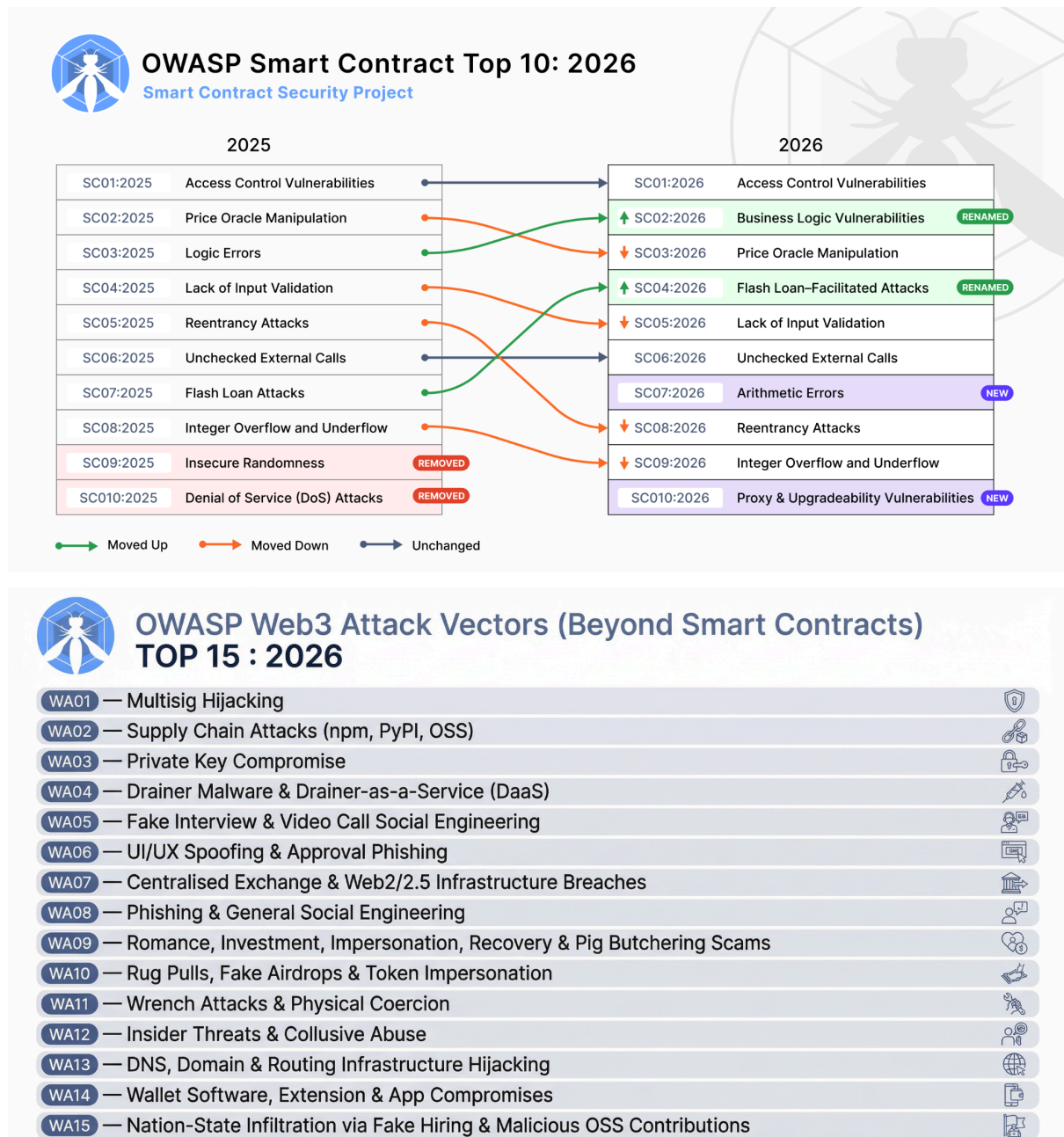
Перевірила:
Селюх П.В.

Київ - 2026

Тема: Дослідження безпечної реалізації та експлуатації децентралізованих додатків.

Мета роботи: отримання навичок роботи із децентралізованими додатками та оцінка безпеки інформації при їх функціонуванні

Під виконання цієї роботи я спирався на



Top 10:2025 List

1. [A01:2025 - Broken Access Control](#)
2. [A02:2025 - Security Misconfiguration](#)
3. [A03:2025 - Software Supply Chain Failures](#)
4. [A04:2025 - Cryptographic Failures](#)
5. [A05:2025 - Injection](#)
6. [A06:2025 - Insecure Design](#)
7. [A07:2025 - Authentication Failures](#)
8. [A08:2025 - Software or Data Integrity Failures](#)
9. [A09:2025 - Security Logging and Alerting Failures](#)
10. [A10:2025 - Mishandling of Exceptional Conditions](#)

[1][OWASP Smart Contract Top 10 : 2026](#)

[2][Alternate Top 15 — Web3 Attack Vectors \(Beyond Smart Contracts\)](#)

[3][OWASP Top 10:2025](#)

Base: Solidity

Як на мене, всі вразливості можна за наслідками оцінити як “критичні”, оскільки, на відміну від простих веб-застосунків тут ми маємо справу з децентралізованими коштами і їх розповсюдженням, використанням і створенням.

Запропоновані мною топ-10:

1. W3-A01: Broken Access Control

Фактор	Значення
Вектор атаки	Використання функцій, з необмеженим доступом для загалу.
Поширеність	Дуже висока.
Складність виявлення	Легко.

Зловмисник, скомпрометований або невмілий користувач може викликати функції, до яких доступу не повинен мати, або обійти неправильну логіку перевірки смартконтрактів. Досить поширена проблема в Solidity. Виявити вразливість легко за допомогою статичних аналізаторів типу Mythril, Slither або вручну. Вплив можна оцінити як критичний, оскільки експлуатація може вести до крадіжки активів, governance takeover або знищення контракту.

В контексті dApps, управління доступом – це процес обмеження виконання функцій лише для певних адрес (EOA) або інших контрактів. У Solidity за замовчуванням усі функції є public. Якщо розробник забуває додати модифікатор (onlyOwner, onlyRole) або використовує небезпечні механізми автентифікації, критичні параметри системи стають вразливими.

Особливістю Web3 є те, що права доступу часто прив'язані до приватних ключів. Якщо логіка контракту дозволяє "перехопити" контекст виклику (наприклад, через tx.origin), зловмисник може діяти від імені адміністратора.

Сценарії експлуатації

Сценарій №1: Несанкціоноване виведення коштів (Unprotected Mint/Withdraw)

Розробник реалізував функцію для виведення накопичених комісій, але забув додати перевірку msg.sender. Хакер викликає withdraw() і вказує свою адресу. Контракт переказує весь баланс зловмиснику.

Сценарій №2: Phishing через tx.origin

Адміністратор контракту використовує require(tx.origin == owner). Хакер надсилає власнику посилання на шкідливий сайт з дропом. Коли власник підписує транзакцію на шкідливому сайті, контракт хакера викликає функцію transferOwnership() у вразливому контракті. Оскільки tx.origin все ще є адресою власника, перевірка проходить, і хакер стає новим власником.

Сценарій №3: Захоплення управління (Governance Takeover)

В системі з багатьма ролями (Admin, Manager, User) функція призначення нових менеджерів доступна самим менеджерам замість

адміна. Компрометація одного ключа менеджера дозволяє зловмиснику створити безліч підконтрольних акаунтів з високими привілеями.

Приклад вразливого коду у Веб3:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

contract ProtocolSettings ____ {
    uint256 public feePercent;
    function updateFee(uint256 _newFee) public {
        require(_newFee <= 100, "Fee too high");
        feePercent = _newFee;
    }
}
```

– можна змінити розмір комісії до 100%

Приклад здорового коду у Веб3:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

import "@openzeppelin/contracts/access/Ownable.sol";
contract ProtocolSettings is Ownable {
    uint256 public feePercent;
    event FeeUpdated(uint256 indexed oldFee, uint256 indexed newFee);
    constructor() Ownable(msg.sender) {}
    function updateFee(uint256 _newFee) public onlyOwner {
        require(_newFee <= 10, "Fee capped at 10%");
        uint256 oldFee = feePercent;
        feePercent = _newFee;
        emit FeeUpdated(oldFee, _newFee);
    }
}
```

– при розгортанні ініціюється власник, а сам модифікатор onlyOwner не дає можливості стороннім користувачам виконати зміну. Функція викличеться тільки з довіреної адреси.

Методи захисту:

1. Explicit Access Control: Використання `onlyOwner` (для простих систем) або `AccessControl` (для складних систем з ролями).
2. Zero-Address Check: При передачі прав власності завжди потрібно перевіряти, щоб нова адреса не була `address(0)`.
3. Visibility Check: Завжди явно вказана видимість функцій (`external`, `public`, `internal`, `private`).
4. `tx.origin`: Використання виключно `msg.sender` для перевірки того, хто викликає функцію.

2. W3-A02: Business Logic & Data Integrity

Фактор	Значення
Вектор атаки	Зловмисник використовує неочевидні послідовності викликів функцій, стани контракту або математичні помилки для отримання непередбаченої вигоди.
Поширеність	Висока.
Складність виявлення	Дуже висока.
Наслідки	Критичні.

У традиційному ПЗ помилка логіки може призвести до збою сервера. У Web3 помилка логіки – це часто "легальна" можливість вивести гроші з контракту, оскільки код – це закон (*Code is Law*). Найвідомішим підвидом цієї категорії є *Reentrancy*, де логіка оновлення балансу відбувається після передачі коштів, що дозволяє зловмиснику вклинитися в процес. Також сюди відносяться помилки округлення та порушення послідовності станів.

Сценарії експлуатації

1. *Reentrancy* : Контракт відправляє ЕТН користувачу. Функція *fallback* у контракті хакера знову викликає функцію зняття коштів до того, як баланс хакера в першому контракті буде встановлено в 0.
2. *Rounding Errors*: У DeFi-протоколах при обміні tokenів з різною кількістю десяткових знаків (наприклад, USDC має 6, а DAI 18).

Зловмисник може здійснювати мікро-транзакції, де через округлення вниз він отримує частку токена безкоштовно.

3. Front-running: Зловмисник бачить транзакцію користувача в mempool і ставить свою транзакцію з вищою комісією, щоб змінити стан ринку перед користувачем.

Методи захисту

1. Патерн CEI (Checks-Effects-Interactions): Завжди змінювати внутрішній стан гаманця і т.д. до того, як взаємодіяти із зовнішніми адресами або іншими контрактами.
2. ReentrancyGuard: Використання стандартних модифікаторів (nonReentrant) від OpenZeppelin для всіх функцій, що виводять кошти.
3. Математика: Для Solidity < 0.8 необхідно використовувати SafeMath. Для всіх версій – будьте уважні при діленні.
4. Інваріанти: Визначення критичних умов системи і тест їх за допомогою інваріантного тестування.

3. W3-A03: Oracle Manipulation & Price Integrity

Фактор	Значення
Вектор атаки	Зловмисник штучно змінює ціну активу на DEX, щоб змусити смартконтракт, який використовує цю ціну, виконати вигідну для хакера дію (наприклад, видати зовелику позику).
Поширеність	Висока
Складність виявлення	Дуже важко
Наслідки	Критичні

Більшість DeFi-протоколів потребують знання поточної ціни активів (наприклад, щоб знати, чи достатньо у користувача застави для позики). Оскільки блокчейн ізольований, він бере ці дані через оракули. Вразливість виникає, коли контракт довіряє одному джерелу ціни, яке

легко змінити (наприклад, поточному резерву пари на Uniswap), або використовує дані, які можна маніпулювати в межах однієї транзакції за допомогою Flash Loans.

Сценарії експлуатації:

1. Spot Price Manipulation: Зловмисник бере величезний Flash Loan, купує токен А за токен В у конкретному пулі. Ціна токена А в цьому пулі миттєво злітає. Вразливий протокол "дивиться" на цей пул, бачить високу ціну і дозволяє хакеру взяти кредит під заставу токена А на суму, що в рази перевищує його реальну вартість.
2. Stale Prices: Оракул перестає оновлюватися, але контракт продовжує використовувати стару ціну під час різкого падіння ринку.
3. L2 Sequencer Offline: У мережах другого рівня, якщо секвенсор падає, ціни в оракулах можуть стати неактуальними, що дозволяє проводити арбітраж проти протоколу.

Методи уникнення:

1. Spot Prices: Відмініть використання reserves або balanceOf децентралізованої біржі як джерело ціни.
2. TWAP (Time-Weighted Average Price): Якщо потрібно використовувати DEX, то краще брати середню ціну за певний проміжок часу. Це робить маніпуляцію в межах однієї транзакції неможливою.
3. Децентралізовані оракули: Рекомендоване використання Chainlink або Pyth. Вони агрегують ціни з багатьох бірж (CEX та DEX), тому маніпуляція на одному майданчику не змінить загальну ціну.
4. Circuit Breakers: Впроваджуйте логіку, яка зупиняє роботу протоколу, якщо ціна змінюється занадто різко.

4. W3-A04: Supply Chain & Infrastructure Security

Фактор	Значення
Вектор атаки	Зловмисник експлуатує вразливості в архітектурі оновлення контрактів

Поширеність	Висхідна.
Складність виявлення	Середня
Наслідки	Критичні

У світі Web3 код не завжди є незмінним. Більшість сучасних dApps використовують патерн Проху для можливості оновлення (upgradeability). Вразливості тут виникають через помилки в управлінні пам'яттю (Storage Collisions), відсутність контролю за ключами адміністраторів, що оновлюють код, або використання застарілих npm-пакетів під час компіляції. Також сюди входить безпека фронтенду, який генерує транзакції для користувачів.

Сценарії експлуатації:

1. Storage Collision: Розробник оновлює логічний контракт через проксі, але змінює порядок оголошення змінних. Оскільки проксі зберігає стан у своїх слотах пам'яті, зміна порядку змінних у новій версії призводить до того, що нові змінні перезаписують старі дані.
2. Compromised Upgrade Keys: Права на виклик upgradeTo() належать одному ЕОА. Хакер отримує приватний ключ через фішинг, оновлює протокол на свій шкідливий контракт і виводить усю ліквідність.
3. Frontend/DNS Hijacking: Смартконтракт повністю безпечний, але хакер захоплює домен dApp (через BGP Hijacking або злом реєстратора). Інтерфейс виглядає так само, але замість виклику функції deposit() він просить користувача підписати approve() на гаманець хакера.

Методи уникнення:

1. Контроль оновлень: Усі оновлення контрактів повинні виконуватися виключно через мультипідпис із часовою затримкою, щоб користувачі могли вивести кошти, якщо оновлення виявиться шкідливим.
2. Захист пам'яті: Під час оновлень варто використовувати плагіни, які автоматично перевіряють код на наявність Storage Collisions перед розгортанням.

3. Неімутабельні залежності: Фіксування версії компілятора та бібліотек (pragma solidity 0.8.20; замість ^0.8.20).
4. Моніторинг інфраструктури: рекомендовано до використання Subresource Integrity для скриптів, надійних DNS-провайдерів з блокуванням змін та регулярного моніторингу SSL-сертифікатів і BGP-маршрутів.

5. W3-A05: Cryptographic Failures & Signature Handling

Фактор	Значення
Вектор атаки	Зловмисник перехоплює криптографічний підпис користувача і зловживає цим
Поширеність	Висока
Складність виявлення	Середня
Наслідки	Критичні

Щоб зекономити газ, багато протоколів дозволяють користувачам підписувати повідомлення off-chain за допомогою свого гаманця, а потім хтось інший (relayer) відправляє цей підпис у смартконтракт.

Смартконтракт використовує функцію есгесовер, щоб дізнатися, хто підписав повідомлення. Якщо розробник не включає у підписані дані унікальні ідентифікатори або використовує вбудовану функцію без перевірок, зловмисник може змусити контракт прийняти підпис двічі або створити фантомний дійсний підпис.

Сценарії експлуатації:

1. Cross-Chain Replay Attack: Користувач підписує транзакцію для мультисігу в мережі Ethereum. Оскільки розробник контракту не включив chain.id у хеш повідомлення, хакер бере цей самий підпис і відправляє його в ідентичний смартконтракт у мережі Polygon, викрадаючи кошти і там.

2. Signature Malleability: Алгоритм ECDSA має математичну особливість: для одного дійсного підпису (v, r, s) можна згенерувати інший, математично еквівалентний підпис (v', r, s') , просто інвертувавши значення s . Якщо контракт захищається від повторного використання просто запам'ятовуючи вже використані підписи у мапінгу (`usedSignatures[signature] = true`), хакер генерує другий підпис і викликає функцію ще раз.
3. Missing Nonce: Користувач підписує чек на переказ 100 токенів. Якщо в підписі немає порядкового номера (`nonce`), хакер може нескінченно відправляти цей чек у контракт, поки не виведе всі гроші користувача.

```
// SPDX-License-Identifier: MIT
```

```
pragma solidity ^0.8.20;
```

```
contract VulnerableToken {
```

```
    mapping(address => uint256) public balances;
```

```
    // ВРАЗЛИВО: Немає nonce, немає chain.id, вразливе до Malleability
```

```
    function transferWithSignature(
```

```
        address to,
```

```
        uint256 amount,
```

```
        bytes32 messageHash,
```

```
        uint8 v, bytes32 r, bytes32 s
```

```
) public {
```

```
    // Відновлюємо адресу з підпису
```

```
    address signer = ecrecover(messageHash, v, r, s);
```

```
    require(signer != address(0), "Invalid signature");
```

```
    require(balances[signer] >= amount, "Insufficient balance");
```

```
    balances[signer] -= amount;
```

```
    balances[to] += amount;
```

```
}
```

```
}
```

```
— незахищений код
```

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

import "@openzeppelin/contracts/utils/cryptography/ECDSA.sol";
import "@openzeppelin/contracts/utils/cryptography/EIP712.sol";
import "@openzeppelin/contracts/utils/Nonces.sol";

contract SecureToken is EIP712, Nonces {
    using ECDSA for bytes32;
    mapping(address => uint256) public balances;

    bytes32 private constant TRANSFER_TYPEHASH =
keccak256("Transfer(address to,uint256 amount,uint256 nonce)");
    constructor() EIP712("SecureToken", "1") {}
    function transferWithSignature(
        address to,
        uint256 amount,
        bytes calldata signature
    ) public {
        address signer = msg.sender;
        bytes32 structHash = keccak256(abi.encode(TRANSFER_TYPEHASH,
to, amount, _useNonce(signer)));
        bytes32 hash = _hashTypedDataV4(structHash);

        address recoveredSigner = hash.recover(signature);

        require(recoveredSigner == signer, "Invalid signature");
        require(balances[signer] >= amount, "Insufficient balance");

        balances[signer] -= amount;
        balances[to] += amount;
    }
}
```

Методи уникнення:

1. Ніколи не пишіть криптографію з нуля: Не використовуйте вбудований `ecrecover`. Завжди використовуйте перевірену бібліотеку

ECDSA.sol від OpenZeppelin, яка містить перевірки на malleability (захист від значень s у верхній половині кривої secp256k1).

2. EIP-712: Це не тільки захищає від Cross-Chain Replay, додаючи chain.id та адресу контракту в хеш, але й дозволяє користувачам бачити в MetaMask зрозумілий текст замість незрозумілого набору байтів. Це найкращий захист від фішингу на рівні фронтенду.
3. Nonces : Кожна дія, що вимагає підпису, повинна містити лічильник nonce, який збільшується на одиницю після кожного успішного виконання. Це робить неможливим повторне використання одного й того ж підпису.

6. W3-A06: Denial of Service (DoS) & Resource Exhaustion

Фактор	Значення
Вектор атаки	Зловмисник створює умови, за яких виконання критичної функції смартконтракту стає неможливим (перевищує Block Gas Limit) або постійно завершується помилкою (revert).
Поширеність	Висока.
Складність виявлення	Очевидно.
Наслідки	Високі.

Блокчейн Ethereum має Block Gas Limit – максимальну кількість обчислень, яку можна виконати в одному блоці. Якщо транзакція вимагає більше газу, ніж дозволено в блоці, вона ніколи не буде виконана.

Вразливість виникає, коли смартконтракт використовує цикли, кількість ітерацій у яких може неконтрольовано зростати, або коли контракт масово розсилає ефір іншим контрактам, один з яких може навмисно відхилити переказ, заблокувавши весь процес.

Сценарії експлуатації

1. Block Gas Limit DoS: Протокол має масив користувачів і функцію, яка проходить по всьому масиву для нарахування дивідендів. Хакер

створює велику кількість порожніх акаунтів і реєструє їх у протоколі. Розмір масиву стає таким великим, що цикл `for` вимагає більше газу, ніж дозволено в блоці. Функція назавжди ламається.

2. **Logical Griefing:** Контракт проводить аукціон або виплачує кошти масиву переможців. Хакер бере участь в аукціоні через смартконтракт, у якому спеціально написана `fallback`-функція, що завжди робить `revert()`. Коли система намагається відправити йому виграш, транзакція відхиляється. Оскільки це відбувається в спільному циклі, гроші не отримує ніхто.

Методи уникнення:

1. **Pull over Push:** Не використовуйте цикли для роздачі токенів і т.д.. Нехай користувачі самі викликають функцію `claim()` або `withdraw()`.
2. **Обмеження розміру масивів:** Якщо цикл абсолютно необхідний, жорстко обмежуйте максимальну кількість ітерацій.
3. **Використання Mappings замість Arrays:** Якщо вам потрібно перевірити наявність елемента, використовуйте `mapping`, що виконується за константний час, замість пошуку по масиву.
4. **Ігнорування невдалих викликів:** Ігноруйте помилку в масових розсилках. Помилку варто реєструвати, але не покладатися на цілковиту успішність циклу.

7. W3-A07: Front-running & MEV

Фактор	Значення
Вектор атаки	Зловмисник моніторить Mempool, знаходить вигідну транзакцію користувача і ставить свою транзакцію раніше, сплативши вищу комісію, щоб отримати економічну вигоду.
Поширеність	Надзвичайно висока.
Складність виявлення	Середня.
Наслідки	Низькі/Середні.

У блокчейні транзакції не виконуються миттєво. Вони потрапляють у Mempool, де майнери бачать їх до того, як включити в блок. Оскільки майнери зазвичай сортують транзакції за ціною газу, будь-хто може перебити транзакцію, запропонувавши більше грошей валідатору.

Це створює можливість для front-running та sandwich attacks, де хакер купує актив перед ціллю, тим самим піднімаючи ціну, і продає його одразу після покупки.

Сценарії експлуатації:

1. Sandwich Attack: Користувач хоче купити токени на певну велику суму. Бот бачить це в Mempool, купує ці токени першим, ціна зростає. Транзакція користувача виконується за завищеною ціною. Бот миттєво продає токени, забираючи різницю собі.
2. Displacement Attack: В системі реєстрації доменних імен або рідкісних активів типу NFT користувач відправляє транзакцію для реєстрації імені "energy.eth". Хакер копіює дані транзакції, ставить вищий газ і реєструє це ім'я на себе на частку секунди раніше.
3. Arbitrage Theft: Користувач знайшов можливість для арбітражу між двома контрактами. Як тільки він відправляє транзакцію, MEV-бот копіює її логіку і виконує цей арбітраж сам – тим самим користувач втрачає прибуток.

Методи уникнення:

1. Strict Slippage: Фронтенд повинен розраховувати amountOutMin на основі поточної ціни мінус 0,5 – 1%. Якщо бот спробує підняти ціну більше ніж на 1%, транзакція жертви просто відхилиться (revert), і хакер втратить газ на свій front-run.
2. Private Transactions: Інтегрувати в dApp можливість відправки транзакцій через Flashbots Protect. Це приховає транзакцію від публічного Mempool, роблячи сендвіч-атаку технічно неможливою.
3. Oracle Checks: Перед обміном звіряти ціну на DEX з ціною в Оракулі. Якщо ціни занадто різняться, блокувати обмін.

8. W3-A08: Insecure Governance & Centralization Risks

Фактор	Значення
Вектор атаки	Зловмисник захоплює контроль над протоколом через маніпуляцію голосуванням або експлуатує бекдори, залишені розробниками під виглядом адмін-функцій.
Поширеність	Висока.
Складність виявлення	Середня.
Наслідки	Критичні.

Більшість Web3-проектів має механізм оновлення або налаштування через голосування власників токенів. Вразливість виникає, коли Flash Loan Governance, тобто можна взяти миттєвий кредит у токенах управління, проголосувати за шкідливу пропозицію і повернути кредит в тій самій транзакції. Centralization Backdoors – розробники залишають за собою право екстреної зупинки або виведення коштів, що робить протокол вразливим до зламу ключа розробника. Якщо для прийняття рішення потрібен низький кворум, зловмисник може непомітно провести атаку у вихідні дні.

Сценарії експлуатації:

1. Governance Takeover: Хакер бачить, що в DAO для прийняття рішення потрібно 4% голосів. Він купує або позичає достатньо токенів, висуває зловмисну пропозицію і миттєво її схвалює.
2. Admin Key Compromise: Протокол має функцію upgradeTo, доступну лише адміну. Адмін – це не Multisig, а звичайний гаманець. Хакер викрадає приватний ключ розробника і замінює код контракту на шкідливий.

Методи уникнення:

1. Snapshots: Голосування повинно базуватися на балансах у минулому, щоб виключити вплив Flash Loans.

2. Timelock: Будь-яке рішення управління повинно ставати активним лише через кілька днів після голосування. Це дає користувачам час вийти з протоколу, якщо рішення шкідливе.
3. Multisig + DAO: Адмінські функції повинні належати не одній людині, а DAO або принаймні Multisig-гаманцю.
4. Guardians: Впровадження ролі охоронців, які можуть скасувати очевидно шкідливу пропозицію, але не можуть приймати власні.

9. W3-A09: Data Privacy & On-Chain Transparency

Фактор	Значення
Вектор атаки	Зловмисник отримує доступ до "приватних" даних смартконтракту, читаючи стан пам'яті безпосередньо з блокчейну, або аналізує транзакції для виявлення конфіденційної інформації.
Поширеність	Висока
Складність виявлення	Легка
Наслідки	Середні/Високі

У Solidity модифікатори `private` та `internal` лише обмежують доступ інших контрактів до цих змінних. Будь-хто може запустити локальний вузол або скористатися інструментами типу `web3.eth.getStorageAt`, щоб прочитати кожен байт даних у пам'яті контракту. Це стає проблемою, коли dApp намагається реалізувати механіки, що потребують прихованості: закриті аукціони або зберігання конфіденційних ключів доступу.

Сценарії експлуатації

1. Reading "Private" Secrets: Контракт зберігає секретне число в змінній `uint256 private adminSecret`. Хакер використовує бібліотеку `Web3.js`, знаходить слот пам'яті, де лежить ця змінна, дізнається число і виграє розіграш.
2. Mempool Information Leakage: Користувач відправляє транзакцію з конфіденційною інформацією. Навіть якщо контракт хешує ці дані,

сама транзакція в Metapool містить вхідні дані в чистому вигляді до моменту включення в блок.

3. MEV Exploitation: Трейдер використовує складний алгоритм для розрахунку арбітражу. Хакер аналізує стан його контракту та параметри, щоб скопіювати стратегію.

Методи уникнення:

1. Assume Public Visibility.
2. Commit-Reveal Schemes
3. Zero-Knowledge Proofs: Для справжньої приватності використовуйте ZK-SNARKs. Це дозволяє довести, що ви знаєте секрет, не розкриваючи самого секрету.
4. Off-chain Computation: Варто обчислювати секретні чи приватні дані поза мережею і передавати в блокчейн лише результат або криптопідтвердження.

10. W3-A10: Insufficient Logging & Monitoring

Фактор	Значення
Вектор атаки	Зловмисник проводить атаку, яка залишається непоміченою через відсутність подій у коді або відсутність систем моніторингу активності.
Поширеність	Достатньо висока.
Складність виявлення	Легка.
Наслідки	Середні/Високі

Без логів зовнішні сервіси не можуть знати, що відбулася важлива дія. Якщо критична функція не генерує подію через економність розробників, зловмисник може діяти непомітно протягом тривалого часу. Це унеможливорює роботу автоматичних запобіжників.

Сценарії експлуатації

1. Silent Drainage: Зловмисник знайшов мікрОВразливість, яка дозволяє виводити по малій частині токєну кожні N хвилин. Оскільки функція

виведення не має emit Withdraw, власники протоколу бачать лише нульовий баланс через добу, коли вже запізно.

2. Unnoticed Governance Attack: Хакер створює шкідливу пропозицію в DAO. Оскільки в системі немає моніторингу подій ProposalCreated, інвестори не отримують сповіщення в Discord/Telegram і не встигають проголосувати "проти".
3. Delayed Exploit Response: На протокол готується атака, що залишає очевидні сліди. Система моніторингу могла б помітити аномальний об'єм токенів, але через відсутність індексації подій реакція відбувається через годину після того, як кошти пішли в міксер.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;
```

```
contract Treasury {
    address public treasuryWallet;
    function updateWallet(address _newWallet) public {
        require(msg.sender == treasuryWallet, "Not authorized");
        treasuryWallet = _newWallet;
    }
}
```

не вразливий код:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;
```

```
contract Treasury {
    address public treasuryWallet;
```

```
    event TreasuryWalletUpdated(address indexed oldWallet, address indexed newWallet);
```

```
    function updateWallet(address _newWallet) public {
        require(msg.sender == treasuryWallet, "Not authorized");
        require(_newWallet != address(0), "Zero address");
```

```
address oldWallet = treasuryWallet;  
treasuryWallet = _newWallet;  
  
emit TreasuryWalletUpdated(oldWallet, _newWallet);  
}  
}
```

Методи уникнення:

1. Log All Critical Changes: Будь-яка зміна балансів, прав доступу або ключових констант повинна супроводжуватися event.
2. Use indexed parameters: Це дозволяє системам моніторингу фільтрувати події за адресою користувача чи типом активу.
3. Real-time Alerting: Налаштування ботів, які сповіщають команду в Slack/Discord про будь-який виклик адмін-функцій або аномально великі транзакції.
4. Off-chain Indexing: Рекомендоване використання The Graph або аналогів для створення надійної бази даних подій dApp.

Висновки:

Під час виконання даної лабораторної роботи були проаналізовані OWASP Top-10, а також основні зазначені ними вразливості для блокчейну і Web3, а також запропонований список Top-10 Web3 dApp. Можна стверджувати, що безпека сучасних децентралізованих додатків трансформувалася з ізольованого аудиту коду смарт-контрактів у комплексну дисципліну, яка охоплює економічну стійкість, цілісність інфраструктури та прозорість управління.

Ефективна стратегія захисту тепер вимагає не лише використання перевірених бібліотек та патернів проєктування, як-от розділення перевірок і взаємодій, а й обов'язкового впровадження децентралізованих оракулів та багаторівневих систем доступу для усунення точок централізації. На відміну від звичайних вебзастосунків, прозорість блокчейну ускладнює розробку dApp, проте в той же час і полегшує її.

Оскільки зловмисники все частіше фокусуються на маніпуляціях ринковими даними та експлуатації архітектури оновлень, розробники повинні віддавати пріоритет принципу найменших привілеїв та використанню часових затримок для критичних операцій.